

数据的发送与接收

北京理工大学计算机学院
金旭亮



本讲内容



在上一讲中，我们介绍了客户端如何连接上服务器，这个，仅仅只是开发网络程序的第一步。



在本讲中，我们将介绍客户端在连接上服务器之后，向服务器发送一个字符串消息，之后马上断开连接。服务器收到客户端发来的消息之后，在控制台窗口之中打印收到的消息。学完本示例，同学们就知道了如何向服务端发送数据。



之后，我们再介绍“反方向的传送”，即服务器向客户端发送数据。



学完本讲，同学们就掌握了在客户端与服务器之间建立双向数据传输通道的基本方法。

Netty网络应用如何发送与接收数据？



Netty中，数据需要放在一个特殊的容器（称为“ByteBuf”）中，之后，再通过“通道（Channel）”发送出去。



对应地，数据接收方也会将“通道”中接收到的数据放在ByteBuf里。



Netty提供了一个Unpooled工具类，可以很方便地创建ByteBuf对象。



ByteBuf是按照字节来存储数据的，因此，所有数据都必须转换为字节，之后才能保存到ByteBuf中，相应地，从ByteBuf中读取的数据，也是字节类型的数据，通常需要进一步转换为其他类型，才能被程序处理。



本讲示例，展示客户端如何将一个字符串发送到服务端。

服务端处理程序

当客户端发送过来数据时，服务端ChannelHandler中的channelRead()方法被回调，从参数（实际上是一个ByteBuf）中可以获取到客户端发来的消息。

```
public class DiscardServerHandler extends ChannelInboundHandlerAdapter {
    @Override
    public void channelActive(ChannelHandlerContext ctx) throws Exception {
        super.channelActive(ctx);
        System.out.println("有客户端连接: " + ctx.channel().remoteAddress());
    }

    @Override
    public void channelRead(ChannelHandlerContext ctx, Object msg) throws Exception {
        ByteBuf buf = (ByteBuf) msg;
        System.out.println("收到消息: [" + buf.toString(CharsetUtil.UTF_8)+" ]");
        //释放资源
        buf.release();
        //收到消息之后，立即断开通道
        ctx.close();
    }
}
```

将ByteBuf中保存的数据按照UTF-8解码为字符串

参看示例ReadAndDestory

服务端主程序

与前一讲示例相比，不同的地方就是使用了不同的ChannelHandler。

```
public class DiscardServer {
    private int port = 9000;
    public void run() throws InterruptedException {
        EventLoopGroup bossGroup = new NioEventLoopGroup();
        EventLoopGroup workerGroup = new NioEventLoopGroup();
        try {
            ServerBootstrap boot = new ServerBootstrap();
            boot.group(bossGroup, workerGroup)
                .channel(NioServerSocketChannel.class)
                .childHandler(new DiscardServerHandler());
            ChannelFuture f = boot.bind(port).addListener(future -> {
                if (future.isSuccess()) {
                    System.out.println(port + "端口绑定成功。");
                } else {
                    System.out.println(port + "端口绑定失败。");
                }
            });
            //采用同步方式阻塞等待服务端通道退出
            f.channel().closeFuture().sync();
        } finally {
            bossGroup.shutdownGracefully();
            workerGroup.shutdownGracefully();
        }
    }

    public static void main(String[] args) throws InterruptedException {
        new DiscardServer().run();
    }
}
```

客户端处理程序

```
public class MyNettyClientHandler extends ChannelInboundHandlerAdapter {  
    @Override  
    public void channelActive(ChannelHandlerContext ctx) throws Exception {  
        var message = LocalDateTime.now() + ":你好, Netty Server!";  
        var messageData = message.getBytes(StandardCharsets.UTF_8);  
        //基于字节数组, 构建一个ByteBuf对象  
        ByteBuf buf = Unpooled.wrappedBuffer(messageData);  
        //将数据发送到服务端  
        ctx.writeAndFlush(buf);  
        //关闭通道  
        ctx.close();  
        System.out.println("数据已经发送, 通道被关闭。\\n" + message);  
    }  
}
```

客户端只要一连接成功, 就立即发送一条消息给服务端, 之后马上关闭通道。

要发送数据, 先把数据放到ByteBuf中, 再调用write()系列方法, 这里要注意, 向通道中写入数据之后, 还需要“flush (冲刷)”, 才会让数据真正地发送出去。之所以这样, 是因为操作系统通常会缓存数据, 等缓存区数据满了才会发送, 想立即发送, 就得通知操作系统不要等了, 立即将现有数据发送出去。

客户端主程序

```
public class MyNettyClient {  
    public static void main(String[] args) throws InterruptedException {  
        NioEventLoopGroup workerGroup = new NioEventLoopGroup();  
        Bootstrap bootstrap=new Bootstrap();  
        bootstrap.group(workerGroup)  
            .channel(NioSocketChannel.class)  
            .handler(new MyNettyClientHandler());  
        var connectFuture= bootstrap.connect("127.0.0.1",9000)  
            .addListener(future -> {  
                if (future.isSuccess()) {  
                    System.out.println("连接成功");  
                }else {  
                    System.out.println("连接失败");  
                }  
            })  
            .sync();  
        //关闭线程池  
        workerGroup.shutdownGracefully().sync();  
        System.out.println("程序退出");  
    }  
}
```

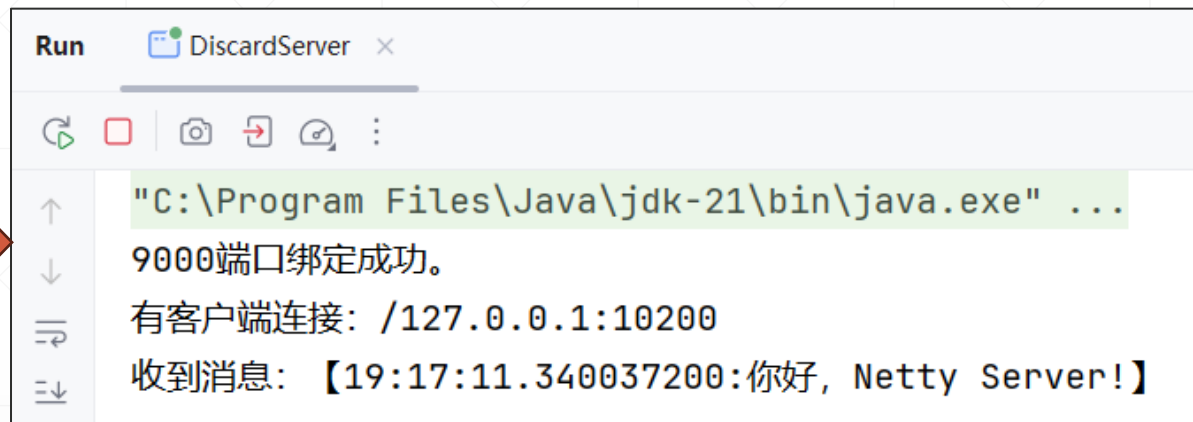
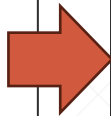
客户端主程序非常简单，就是直接设定使用前面介绍的ChannelHandler来发送信息。

运行截图



Run DiscardServer x MyNettyClient x

```
"C:\Program Files\Java\jdk-21\bin\java.exe" ...  
连接成功  
数据已经发送, 通道被关闭。  
19:17:11.340037200:你好, Netty Server!  
程序退出  
  
Process finished with exit code 0
```



Run DiscardServer x

```
"C:\Program Files\Java\jdk-21\bin\java.exe" ...  
9000端口绑定成功。  
有客户端连接: /127.0.0.1:10200  
收到消息: 【19:17:11.340037200:你好, Netty Server!】
```

通过本讲示例, 你已经知道了客户端可以调用writeAndFlush方法将数据发给服务器, 相应地, 服务端也可以调用这个方法将数据发给客户端, 从而构建出客户端与服务端“双向通讯”的数据通道。请看下一示例 (SendAndRecv)

Server端处理程序

当一有客户端连接，服务端就向客户端发送一条欢迎消息

```
public class MyServerHandler extends ChannelInboundHandlerAdapter {  
    @Override  
    public void channelActive(ChannelHandlerContext ctx) throws Exception {  
        System.out.println("有客户端连接: "+ctx.channel().remoteAddress());  
        var welcomeMessage="欢迎你访问本服务器!".getBytes(StandardCharsets.UTF_8);  
        ctx.writeAndFlush(Unpooled.wrappedBuffer(welcomeMessage));  
    }  
  
    @Override  
    public void channelRead(ChannelHandlerContext ctx, Object msg) throws Exception {  
        ByteBuf buf = (ByteBuf) msg;  
        System.out.println("收到数据: "+buf.toString(StandardCharsets.UTF_8));  
    }  
  
    @Override  
    public void channelInactive(ChannelHandlerContext ctx) throws Exception {  
        System.out.println("客户端断开连接: "+ctx.channel().remoteAddress());  
    }  
}
```

在SendAndRecv示例的服务端处理程序中，重写了基类的三个方法，channelActive()方法前面已经见过了，channelRead()方法用于读以客户端传来的消息，而channelInactive()则会在客户端断开连接时被调用。

客户端处理程序—发送数据

```
public class MyClientSendHandler extends ChannelInboundHandlerAdapter {  
    @Override  
    public void channelActive(ChannelHandlerContext ctx) throws Exception {  
        System.out.println(new Date() + ":客户端发送数据");  
        var messageData = "你好, Netty Server".getBytes(StandardCharsets.UTF_8);  
        //将数据发送到服务端  
        ctx.writeAndFlush(Unpooled.wrappedBuffer(messageData));  
    }  
}
```

从示例中可以看到，要发送消息，就把消息放到ByteBuf中，然后再调用writeAndFlush()方法就行了。这个是通用作法。

客户端处理程序-接收数据

数据的接收，也是“固定套路”，就是在ChannelHandler的相关方法（通常是channelReader方法中，通过msg参数进行接收：

```
public class MyClientReceiveHandler extends ChannelInboundHandlerAdapter {  
    @Override  
    public void channelRead(ChannelHandlerContext ctx, Object msg) throws Exception {  
        var receivedMsg = ((ByteBuf) msg).toString(StandardCharsets.UTF_8);  
        System.out.println("收到数据: " + receivedMsg);  
        System.out.println("敲回车键退出.....");  
    }  
}
```

使用管线组合两个处理程序

```
public class MyNettyClient {  
    public static void main(String[] args) {  
        NioEventLoopGroup workerGroup = new NioEventLoopGroup();  
        Bootstrap bootstrap=new Bootstrap();  
        bootstrap.group(workerGroup)  
            .channel(NioSocketChannel.class)  
            .handler(new ChannelInitializer<SocketChannel>() {  
                @Override  
                protected void initChannel(SocketChannel ch) throws Exception {  
                    //添加handler将数据发送到Server端  
                    ch.pipeline().addLast(new MyClientSendHandler());  
                    ch.pipeline().addLast(new MyClientReceiveHandler());  
                }  
            });  
        bootstrap.connect("127.0.0.1",9000).addListener(future -> {  
            if (future.isSuccess()) {  
                System.out.println("连接成功");  
            }else {  
                System.out.println("连接失败");  
            }  
        });  
  
        new Scanner(System.in).nextLine();  
        workerGroup.shutdownGracefully();  
    }  
}
```

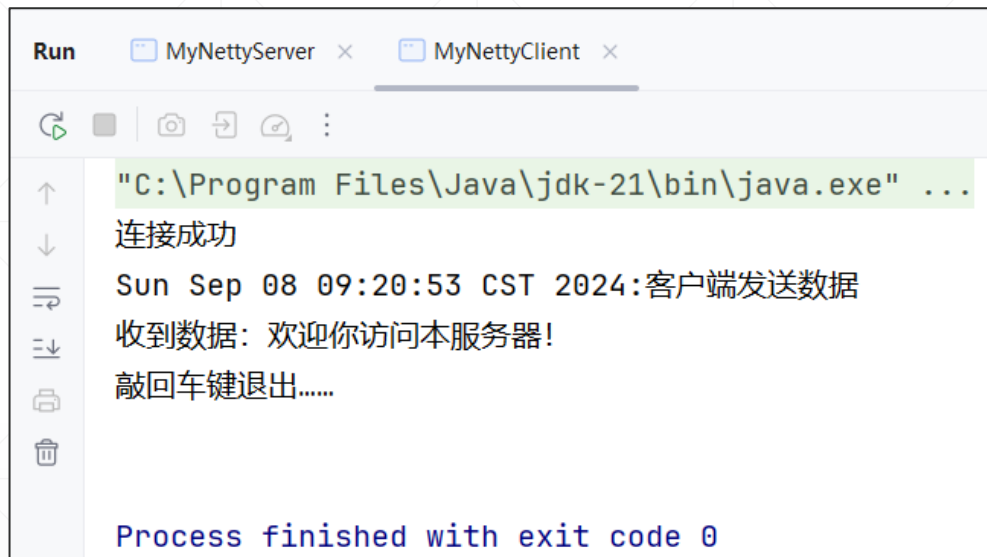
与前面的例子不一样，本例的客户端有两个Handler，为此，需要把这两个Handler“接起来”，构成一条“数据处理流水线”。

构建“数据处理流水线”的方法也是固定的，就是创建一个匿名类继承

ChannelInitialize基类，然后，重写initChannel()方法，在这里面写代码向管线中追加Handler。

ChannelPipeline很重要，在后面的课程中会专门介绍它。

运行截图



```
Run  MyNettyServer x  MyNettyClient x
"C:\Program Files\Java\jdk-21\bin\java.exe" ...
连接成功
Sun Sep 08 09:20:53 CST 2024:客户端发送数据
收到数据: 欢迎你访问本服务器!
敲回车键退出.....

Process finished with exit code 0
```



```
Run  MyNettyServer x
"C:\Program Files\Java\jdk-21\bin\java.exe" ...
服务器启动完毕.....
有客户端连接: /127.0.0.1:2665
收到数据: 你好, Netty Server
客户端断开连接: /127.0.0.1:2665

Process finished with exit code 130
```

至此为止，同学们应该已经学会了如何在服务端与客户端“双向”发送和接收消息。